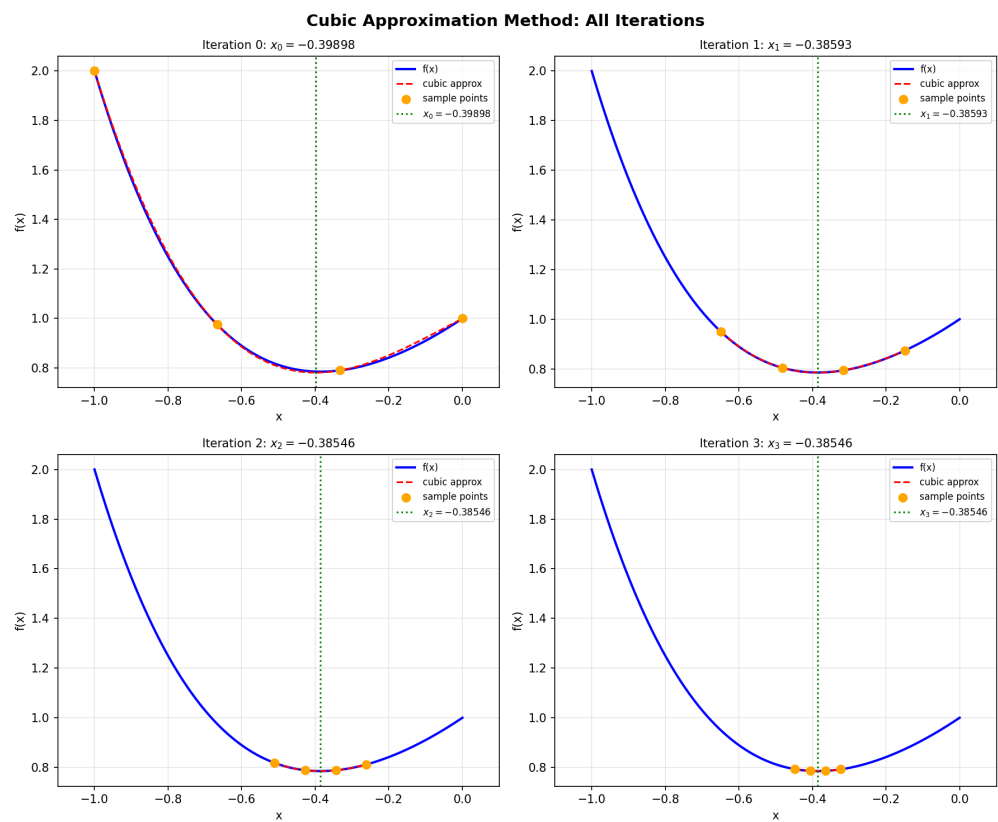
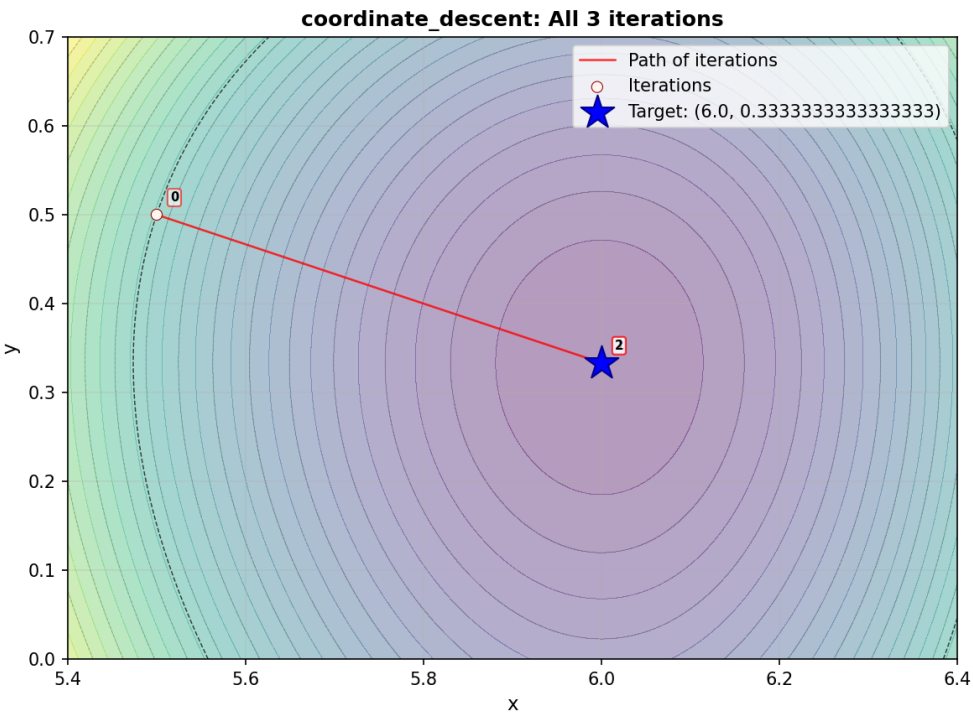


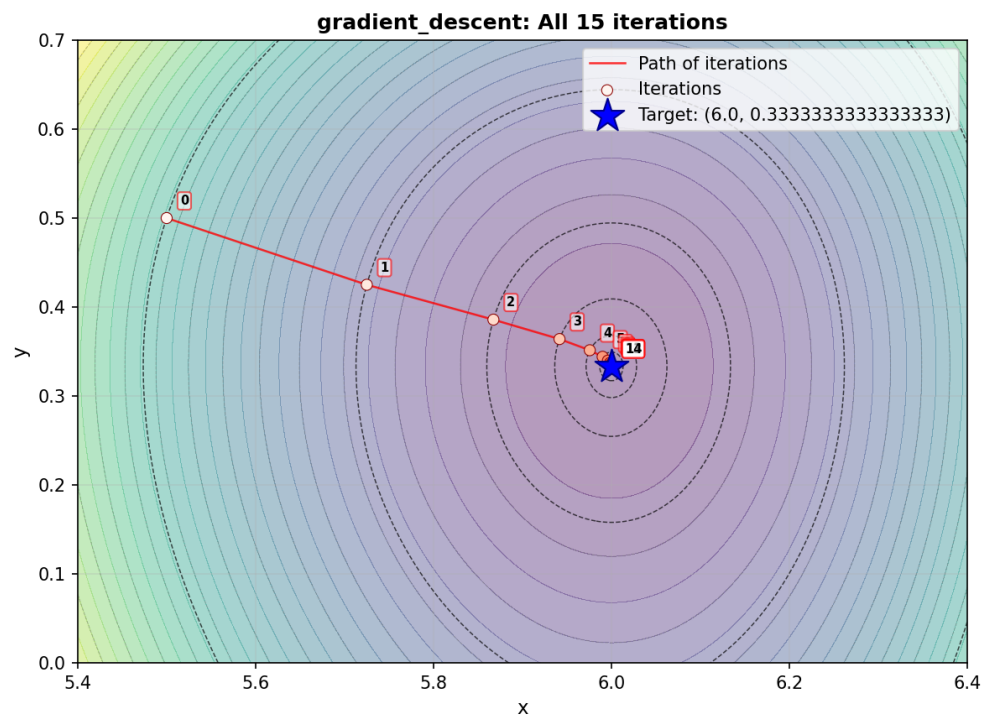
Графики



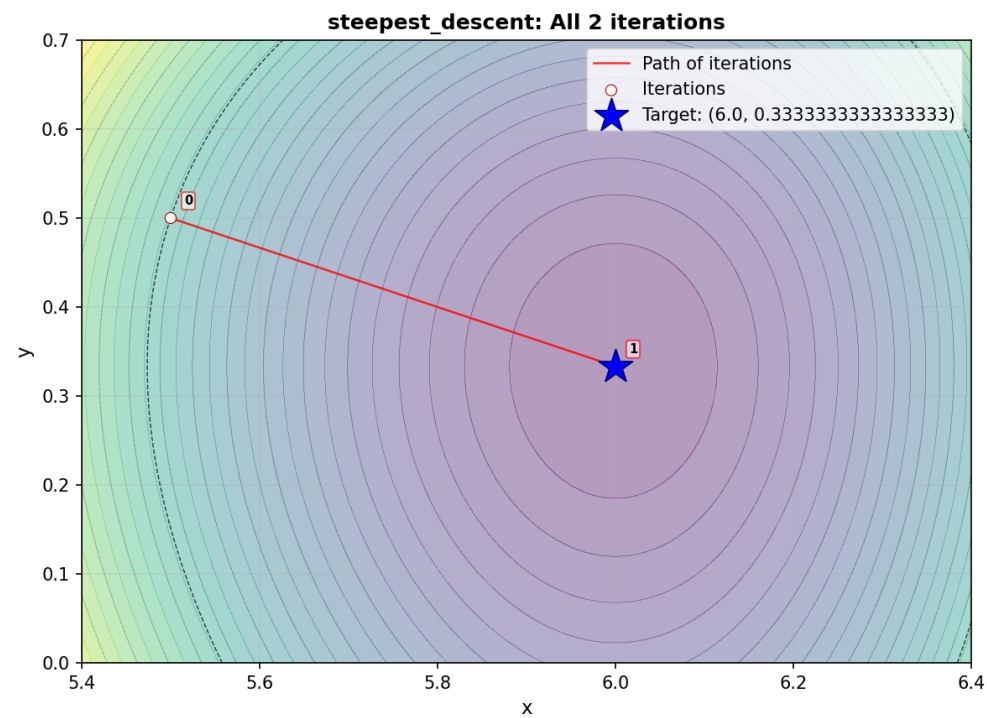
Метод кубической аппроксимации



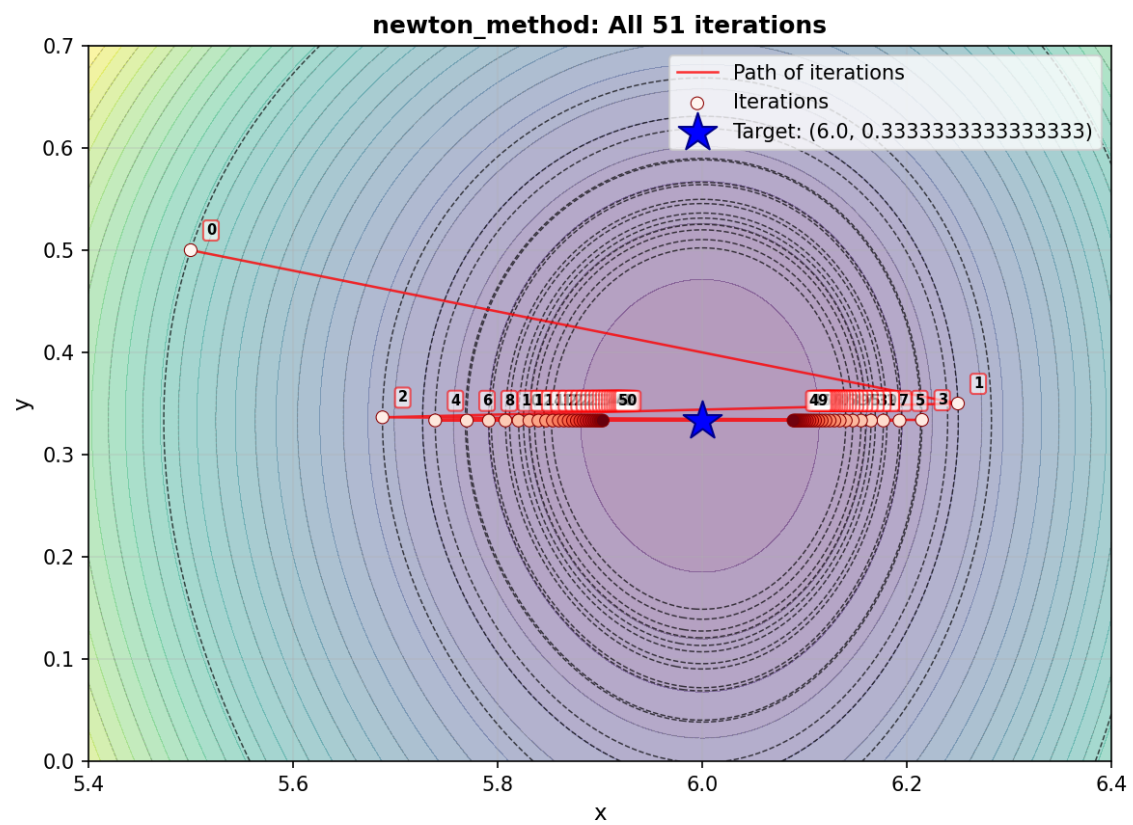
Покоординатный спуск



Градиентный спуск



Наискорейший спуск



Метод Ньютона

Листинг: model.py

Листинг 1: Объектная модель методов оптимизации

```
1 from dataclasses import dataclass
2 from typing import Callable, List, Tuple
3 import math
4 import numpy as np
5
6
7 def _golden_section(f: Callable[[float], float], a: float, b: float, tol: float = 1e-5, maxiter: int = 100) -> float:
8     gr = (math.sqrt(5) - 1) / 2
9     c = b - gr * (b - a)
10    d = a + gr * (b - a)
11    try:
12        fc = f(c)
13    except Exception:
14        fc = float('inf')
15    try:
16        fd = f(d)
17    except Exception:
18        fd = float('inf')
19    for _ in range(maxiter):
20        if math.isnan(fc) or math.isinf(fc):
21            fc = float('inf')
22        if math.isnan(fd) or math.isinf(fd):
23            fd = float('inf')
24        if fc < fd:
25            b = d
26            d = c
27            fd = fc
28            c = b - gr * (b - a)
29            try:
30                fc = f(c)
31            except Exception:
32                fc = float('inf')
33        else:
34            a = c
35            c = d
36            fc = fd
37            d = a + gr * (b - a)
38            try:
39                fd = f(d)
40            except Exception:
41                fd = float('inf')
42        if abs(b - a) < tol:
43            break
44    return (a + b) / 2
45
```

```

46
47 class Function2D:
48     def __init__(self, f: Callable[[np.ndarray], float], grad: Callable[[np.ndarray], np.ndarray] = None):
49         self.f = f
50         self.grad = grad
51
52     def value(self, xy: np.ndarray) -> float:
53         return float(self.f(xy))
54
55     def gradient(self, xy: np.ndarray) -> np.ndarray:
56         if self.grad is None:
57             eps = 1e-8
58             x, y = xy
59             dx = (self.f(np.array([x + eps, y])) - self.f(np.array([x - eps, y]))) / (2 * eps)
60             dy = (self.f(np.array([x, y + eps])) - self.f(np.array([x, y - eps]))) / (2 * eps)
61             return np.array([dx, dy])
62         return np.array(self.grad(xy))
63
64
65 @dataclass
66 class Result2D:
67     xs: List[Tuple[float, float]]
68     fs: List[float]
69
70
71 class Optimizer2D:
72     def __init__(self, func: Function2D, x0: Tuple[float, float], eps: float = 1e-4, maxiter: int = 200):
73         self.func = func
74         self.x0 = np.array(x0, dtype=float)
75         self.eps = eps
76         self.maxiter = maxiter
77
78     def optimize(self) -> Result2D:
79         raise NotImplementedError
80
81
82 class CoordinateDescent(Optimizer2D):
83     def optimize(self) -> Result2D:
84         x = self.x0.copy()
85         xs = [tuple(x)]
86         fs = [self.func.value(x)]
87         for _ in range(self.maxiter):
88             def phi_x(t):
89                 return self.func.value(np.array([t, x[1]]))
90             a, b = x[0] - 2, x[0] + 2
91             x[0] = _golden_section(phi_x, a, b, tol=1e-5)
92             def phi_y(t):
93                 return self.func.value(np.array([x[0], t]))
94             a, b = x[1] - 2, x[1] + 2

```

```

95     x[1] = _golden_section(phi_y, a, b, tol=1e-5)
96     xs.append(tuple(x))
97     fs.append(self.func.value(x))
98     if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
99         break
100     return Result2D(xs, fs)
101
102
103 class GradientDescent(Optimizer2D):
104     def optimize(self) -> Result2D:
105         x = self.x0.copy()
106         xs = [tuple(x)]; fs = [self.func.value(x)]
107         alpha0 = 0.1
108         for _ in range(self.maxiter):
109             g = self.func.gradient(x)
110             if np.linalg.norm(g) < self.eps:
111                 break
112             alpha = alpha0
113             c = 1e-4; rho = 0.5
114             fx = self.func.value(x)
115             for _ in range(60):
116                 xn = x - alpha * g
117                 fn = self.func.value(xn)
118                 if fn <= fx - c * alpha * np.dot(g, g):
119                     break
120                 alpha *= rho
121             x = x - alpha * g
122             xs.append(tuple(x)); fs.append(self.func.value(x))
123             if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
124                 break
125             return Result2D(xs, fs)
126
127
128 class SteepestDescent(Optimizer2D):
129     def optimize(self) -> Result2D:
130         x = self.x0.copy()
131         xs = [tuple(x)]; fs = [self.func.value(x)]
132         for _ in range(self.maxiter):
133             g = self.func.gradient(x)
134             if np.linalg.norm(g) < self.eps:
135                 break
136             def phi(a):
137                 return self.func.value(x - a * g)
138             alpha = _golden_section(phi, 0.0, 10.0, tol=1e-5)
139             x = x - alpha * g
140             xs.append(tuple(x)); fs.append(self.func.value(x))
141             if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
142                 break
143             return Result2D(xs, fs)

```

Листинг: `runmodel.py`

Листинг 2: Запуск объектной модели

```
1 from .model import Function2D, CoordinateDescent, GradientDescent, SteepestDescent
2 import numpy as np
3 from pathlib import Path
4 import json
5
6 def f(xy):
7     x, y = xy
8     return x**3 - 15*x**2 + 72*x + y**3 + y**2 - y - 113
9
10 def grad(xy):
11     x, y = xy
12     return np.array([3*x**2 - 30*x + 72, 3*y**2 + 2*y - 1])
13
14 def main():
15     base = Path(__file__).resolve().parent
16     figures = base.parent / 'figures'
17     figures.mkdir(exist_ok=True)
18     func = Function2D(f, grad)
19     x0 = (5.5, 0.5)
20     cd = CoordinateDescent(func, x0, eps=1e-4, maxiter=200).optimize()
21     gd = GradientDescent(func, x0, eps=1e-4, maxiter=200).optimize()
22     sd = SteepestDescent(func, x0, eps=1e-4, maxiter=200).optimize()
23     res = {
24         'coordinate_iters': len(cd.xs),
25         'gradient_iters': len(gd.xs),
26         'steepest_iters': len(sd.xs),
27         'coordinate_last': cd.xs[-1],
28         'gradient_last': gd.xs[-1],
29         'steepest_last': sd.xs[-1]
30     }
31     (base / 'results_model.json').write_text(json.dumps(res, indent=2))
32
33 if __name__ == '__main__':
34     main()
```